

```

        this.Controls.Add(input);
    }

    public static void Test()
    {
        Application.Run(new SlowForm());
    }
}

```

While we do the button's work, the UI might become unresponsive. This is because the single thread of execution that processes events on our form is tied up executing the event handler for the button click. In these cases where we know our engine processing will take a long time, we would like to avoid hanging the UI by executing the engine work in a different thread.

The thread class in C# provides this facility. We can instantiate a new thread with a delegate (of type `ThreadStart` with signature `void()`) that the thread will run and then start it with the `Start()` method.

This thread operates concurrently with the original thread of execution. So now, this original thread of execution is able to continue processing events fired by the UI while the new thread handles the expensive computation.

```

public class ConcurrentForm : Form
{
    Button button;
    TextBox input;
    public ConcurrentForm()
    {
        Size = new Size(200, 200);

        input = new TextBox();
        input.Size = new Size(100, 25);
        input.Location = new Point(35, 50);

        button = new Button();
        button.Size = new Size(100, 25);
        button.Location = new Point(35, 100);
        button.Text = "Do Work";
        button.Click += new EventHandler(delegate(object
sender, EventArgs e)
        {

```